

Отчёт по лабораторной работе №7

- Цигельник Юля Б9124-09.03.03пикд(3)
- <https://github.com/yunohaha/vectorengine/tree/lab-7>

Базовый уровень

Выбор объекта

Выбор объекта реализован через hit-testing - определение попадания курсора в область фигуры.

```
const findShapeAtPoint = useCallback((point: Point2D): Shape | null => {
  const currentShapes = shapesRef.current;
  for (let i = currentShapes.length - 1; i >= 0; i--) {
    if (currentShapes[i].hitTest(point.x, point.y)) {
      return currentShapes[i];
    }
  }
  return null;
}, []);
```

- При клике мыши вычисляются координаты курсора
- Происходит обход всех фигур в обратном порядке отрисовки (с верхнего слоя к нижнему)
- Для каждой фигуры вызывается метод hitTest(), который определяет, находится ли точка внутри фигуры
- При обнаружении фигуры она возвращается и становится выбранной

Координатные системы

```

const getMouseCoords = (e: React.MouseEvent): Point2D => {
  if (!canvasRef.current) return { x: 0, y: 0 };
  const rect = canvasRef.current.getBoundingClientRect();
  const scaleX = canvasRef.current.width / rect.width;
  const scaleY = canvasRef.current.height / rect.height;
  return {
    x: (e.clientX - rect.left) * scaleX,
    y: (e.clientY - rect.top) * scaleY
  };
};

```

Здесь происходит преобразование из координат окна браузера в координаты canvas с учетом масштабирования

Перемещение объекта

```

if (dragType.current === 'move') {
  shape.transform.x = start.x + dx;
  shape.transform.y = start.y + dy;
  if (onShapesChange) {
    onShapesChange([...shapesRef.current]);
  }
  return;
}

```

- При начале перетаскивания сохраняется начальная позиция объекта (dragStartTransform)
- При движении мыши вычисляется смещение (dx, dy)
- Новая позиция = начальная позиция + смещение
- Объект следует за курсором без рывков благодаря использованию requestAnimationFrame для плавной отрисовки

Корректная работа координат

Используются 4 системы координат:

1. Координаты окна браузера - полученные из события мыши e.clientX/e.clientY
2. Координаты canvas - преобразованные с учетом размера canvas (функция getMouseCoords)
3. Локальные координаты фигуры - относительные координаты внутри фигуры (метод transformPointToLocal)
4. Мировые координаты - абсолютные координаты на сцене

Преобразование между системами выполняется с помощью матриц трансформации:

```
// QuadraticBezier.ts
getDevicePoints(): Point2D[] {
  const matrix = this.getLocalToDeviceMatrix();
  return this.flattenLocal().map(p => mat3.transformPoint(matrix, p.x, p.y));
}
```

Изменение размеров

Определение ручек масштабирования

```
const getHandles = useCallback((shape: Shape) => {
  const bounds = shape.getBounds();
  const center = shape.getCenter();
  return {
    se: { x: bounds.maxX, y: bounds.maxY },
    nw: { x: bounds.minX, y: bounds.minY },
    ne: { x: bounds.maxX, y: bounds.minY },
    sw: { x: bounds.minX, y: bounds.maxY },
    rotate: { x: center.x, y: bounds.minY - ROTATE_HANDLE_DISTANCE }
  };
}, []);
```

Hit-тестирование ручек

```
const hitTestHandle = useCallback((shape: Shape, point: Point2D): string | null => {
  const handles = getHandles(shape);
  const hitZone = HANDLE_SIZE + 5;

  if (Math.abs(point.x - handles.se.x) <= hitZone && Math.abs(point.y - handles.se.y) <= hitZone) return 'se';
  if (Math.abs(point.x - handles.nw.x) <= hitZone && Math.abs(point.y - handles.nw.y) <= hitZone) return 'nw';
  if (Math.abs(point.x - handles.ne.x) <= hitZone && Math.abs(point.y - handles.ne.y) <= hitZone) return 'ne';
  if (Math.abs(point.x - handles.sw.x) <= hitZone && Math.abs(point.y - handles.sw.y) <= hitZone) return 'sw';
  if (Math.abs(point.x - handles.rotate.x) <= hitZone && Math.abs(point.y - handles.rotate.y) <= hitZone) return 'rotate';

  return null;
}, [getHandles]);
```

Логика масштабирования

```

iif (dragType.current === 'resize' && dragHandle.current) {
  const sensitivity = 0.02;
  const minScale = 0.5;
  const handle = dragHandle.current;

  switch (handle) {
    case 'se':
      shape.transform.scaleX = Math.max(minScale, start.scaleX + dx * sensitivity);
      shape.transform.scaleY = Math.max(minScale, start.scaleY + dy * sensitivity);
      break;
    case 'nw':
      shape.transform.scaleX = Math.max(minScale, start.scaleX - dx * sensitivity);
      shape.transform.scaleY = Math.max(minScale, start.scaleY - dy * sensitivity);
      shape.transform.x = start.x + dx / 2;
      shape.transform.y = start.y + dy / 2;
      break;
    case 'ne':
      shape.transform.scaleX = Math.max(minScale, start.scaleX + dx * sensitivity);
      shape.transform.scaleY = Math.max(minScale, start.scaleY - dy * sensitivity);
      shape.transform.y = start.y + dy / 2;
      break;
    case 'sw':
      shape.transform.scaleX = Math.max(minScale, start.scaleX - dx * sensitivity);
      shape.transform.scaleY = Math.max(minScale, start.scaleY + dy * sensitivity);
      shape.transform.x = start.x + dx / 2;
      break;
  }
  if (onShapesChange) {
    onShapesChange([...shapesRef.current]);
  }
}
}, [onShapesChange]);

```

Поворот

```
if (dragType.current === 'rotate') {  
  const center = shape.getCenter();  
  const startAngle = Math.atan2(dragStart.current.y - center.y, dragStart.current.x - center.x);  
  const currentAngle = Math.atan2(point.y - center.y, point.x - center.x);  
  shape.transform.rotation = start.rotation + (currentAngle - startAngle);  
  if (onShapesChange) {  
    onShapesChange([...shapesRef.current]);  
  }  
  return;  
}
```

- Вычисляется угол между центром фигуры и курсором в начальный момент (startAngle)
- Вычисляется текущий угол между центром фигуры и курсором (currentAngle)
- Используется функция $\text{atan2}(y, x)$, которая возвращает угол в радианах от $-\pi$ до π
- Новый угол поворота = начальный угол фигуры + разница между текущим и стартовым углами
- Это позволяет вращать объект на 360 градусов без скачков

Визуальные ручки управления

```
const frame = () => {
  const r = rendererRef.current;
  if (r) {
    r.beginFrame(true);

    // Отрисовка всех фигур
    for (const shape of shapesRef.current) {
      shape.drawRaster(r);
    }

    const currentSelectedId = selectedIdRef.current;
    if (currentSelectedId) {
      const selectedShape = shapesRef.current.find(s => s.id === currentSelectedId);
      if (selectedShape) {
        const bounds = selectedShape.getBounds();
        const outlineColor = { r: 41, g: 34, b: 77, a: 200 };

        // Рамка выделения
        r.strokeLine(bounds.minX, bounds.minY, bounds.maxX, bounds.minY, outlineColor, 1);
        r.strokeLine(bounds.maxX, bounds.minY, bounds.maxX, bounds.maxY, outlineColor, 1);
        r.strokeLine(bounds.maxX, bounds.maxY, bounds.minX, bounds.maxY, outlineColor, 1);
        r.strokeLine(bounds.minX, bounds.maxY, bounds.minX, bounds.minY, outlineColor, 1);

        const center = selectedShape.getCenter();
        const handleColor = { r: 41, g: 34, b: 77, a: 200 };
        const rotateColor = { r: 106, g: 90, b: 205, a: 255 };

        // Ручки масштабирования (по углам)
        r.fillCircle(bounds.maxX, bounds.maxY, HANDLE_SIZE, handleColor);
        r.fillCircle(bounds.minX, bounds.minY, HANDLE_SIZE, handleColor);
        r.fillCircle(bounds.maxX, bounds.minY, HANDLE_SIZE, handleColor);
        r.fillCircle(bounds.minX, bounds.maxY, HANDLE_SIZE, handleColor);

        // Ручка поворота (над объектом)
        r.fillCircle(center.x, bounds.minY - ROTATE_HANDLE_DISTANCE, HANDLE_SIZE, rotateColor);
      }
    }
    r.commit();
  }
};
```

- Синяя рамка толщиной 2 пикселя вокруг выбранного объекта
- 4 угловые ручки (белые круги) для масштабирования
- Красная ручка над объектом для поворота

Корректная работа с несколькими объектами

Управление слоями

```
const moveLayer = useCallback((id: string, direction: 'up' | 'down') => {
  setShapes(prev => {
    const index = prev.findIndex(s => s.id === id);
    if (index === -1) return prev;
    const newShapes = [...prev];
    if (direction === 'up' && index < prev.length - 1) {
      [newShapes[index], newShapes[index + 1]] = [newShapes[index + 1], newShapes[index]];
    } else if (direction === 'down' && index > 0) {
      [newShapes[index], newShapes[index - 1]] = [newShapes[index - 1], newShapes[index]];
    }
    return newShapes;
  });
}, []);
```

Визуальное представление слоев в UI:

```

<div className="layer-list">
  {shapes.map((shape, idx) => (
    <div
      key={shape.id}
      className={`layer-item ${selectedId === shape.id ? 'active' : ''}`}
      onClick={() => setSelectedId(shape.id)}
    >
      <span className="layer-index">{idx + 1}</span>
      <span className="layer-name">{shape.constructor.name.slice(0, 8)}</span>
      <div className="layer-buttons">
        <button onClick={(e) => { e.stopPropagation(); moveLayer(shape.id, 'up'); }}>
          ↑
        </button>
        <button onClick={(e) => { e.stopPropagation(); moveLayer(shape.id, 'down'); }}>
          ↓
        </button>
      </div>
    </div>
  )
  )
)}
</div>

```

- Массив shapes хранит объекты в порядке отрисовки (индекс 0 - нижний слой)
- При клике выбор происходит с верхнего слоя (обход массива в обратном порядке)
- Кнопки ↑ и ↓ позволяют менять порядок слоев
- Выбранный объект подсвечивается в списке слоев

Оптимальный уровень

Контрольные точки (ControlPointsEditor.ts)

```
export class ControlPointsEditor {
    private static readonly POINT_SIZE = 6;

    static getControlPoints(shape: Shape): Point2D[] {
        if (typeof (shape as any).getControlPoints === 'function') {
            return (shape as any).getControlPoints();
        }
        if ((shape as any).points) {
            return (shape as any).points;
        }
        return [];
    }

    static hitTest(shape: Shape, point: Point2D): { index: number; point: Point2D } | null {
        const points = this.getControlPoints(shape);
        if (points.length === 0) return null;

        for (let i = 0; i < points.length; i++) {
            try {
                const devicePoint = shape.transformPointToDevice(points[i].x, points[i].y);
                const dx = Math.abs(point.x - devicePoint.x);
                const dy = Math.abs(point.y - devicePoint.y);
                if (dx <= this.POINT_SIZE + 5 && dy <= this.POINT_SIZE + 5) {
                    return { index: i, point: points[i] };
                }
            } catch (e) {
                console.warn('HitTest error for point', i, e);
            }
        }
        return null;
    }

    static draw(r: RasterRenderer, shape: Shape): void {
        const points = this.getControlPoints(shape);
        if (points.length === 0) return;

        for (let i = 0; i < points.length; i++) {
            try {
```

```

        const devicePoint = shape.transformPointToDevice(points[i].x, points[i].y);

        r.fillCircle(devicePoint.x, devicePoint.y, this.POINT_SIZE + 2, { r: 255, g: 255, b: 255 });

        r.fillCircle(devicePoint.x, devicePoint.y, this.POINT_SIZE, { r: 46, g: 204, b: 255 });
        r.strokeCircle(devicePoint.x, devicePoint.y, this.POINT_SIZE, { r: 0, g: 0, b: 0 });
      } catch (e) {
        console.warn('Draw error for point', i, e);
      }
    }
  }
}

```

Редактирование контрольных точек (CanvasScene.tsx):

```

if (dragType.current === 'control-point' && dragControlPointIndex.current !== null) {
  const index = dragControlPointIndex.current;
  const points = ControlPointsEditor.getControlPoints(shape);
  if (points && points[index]) {
    // Переводим экранные координаты в локальные
    const localPoint = shape.transformPointToLocal(point.x, point.y);
    points[index] = localPoint;

    // Обновляем фигуру
    if ('setControlPoint' in shape) {
      (shape as any).setControlPoint(index, localPoint);
      if (onShapesChange) {
        onShapesChange([...shapesRef.current]);
      }
    }
  }
}
return;
}

```

Реализация getControlPoints для различных фигур:

```

getControlPoints(): Point2D[] {
  return this.getLocalCorners(); // Возвращает 4 угловые точки
}

```

Доп функции

Удаление объекта

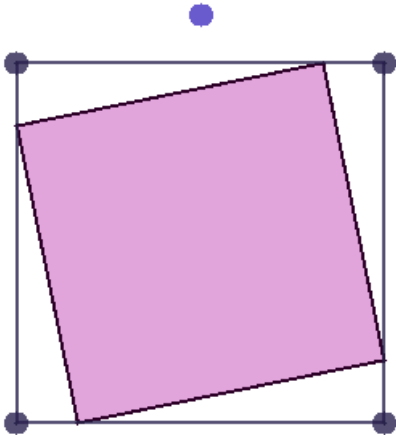
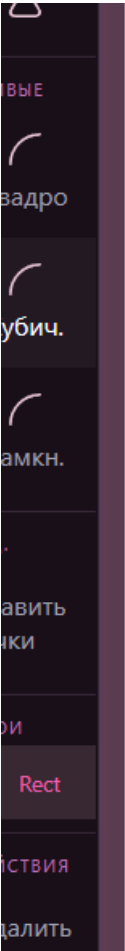
```
const deleteShape = useCallback((id: string) => {
  setShapes(prev => prev.filter(shape => shape.id !== id));
  if (selectedId === id) {
    setSelectedId(null);
    setEditingShapeId(null);
  }
}, [selectedId]);

const deleteSelected = () => {
  if (selectedId) {
    deleteShape(selectedId);
  }
};
```

Горячие клавиши

```
const handleKeyDown = useCallback((e: KeyboardEvent) => {
  if (e.key === 'Delete' && selectedIdRef.current && onShapesChange) {
    const newShapes = shapesRef.current.filter(s => s.id !== selectedIdRef.current);
    shapesRef.current = newShapes;
    onShapesChange(newShapes);
    onSelect?.(null);
  }
  if (e.key === 'Escape' && onSelect) {
    onSelect(null);
  }
}, [onShapesChange, onSelect]);
```

Скриншоты



Свойства

Тип
Rect

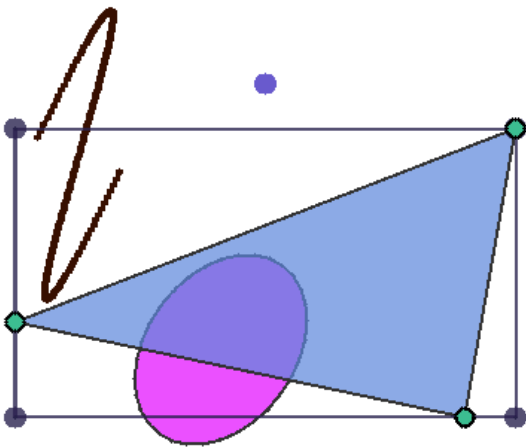
Позиция X
348

Позиция Y
461

Поворот
348°

Масштаб X
1.62

Масштаб Y
2.36



Тип
Triangle

Позиция X
852

Позиция Y
318

Поворот
147°

Масштаб X
1.00

Масштаб Y
1.00

Режим правки
Перетаскивайте зеленые точки



ИВЫЕ



Свадро



Кубич.



Замкн.

д.

ОДВИЖАТЬ

